DSLab. 02 Vivado Block Design & Verilog

Lab. 02 Vivado Block Design & Verilog

- Vivado Block Design
- Verilog
- ~~Please write and upload the lab report (Lab02) -- Due on 2023/09/15 23:59~~

Binary Logic

- **Binary logic** consists of **binary variables** and **a set of logical operations**
- There are three basic logical operations: **AND**, **OR**, and **NOT**

Table 1.8
Truth Tables of Logical Operations

AND

x	y	$x \cdot y$
0	0	0
0	1	0
1	0	0
1	1	1

OR

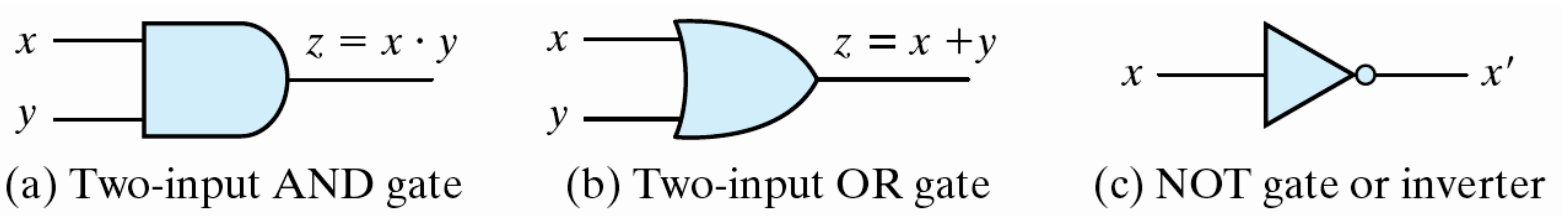
x	y	$x + y$
0	0	0
0	1	1
1	0	1
1	1	1

NOT

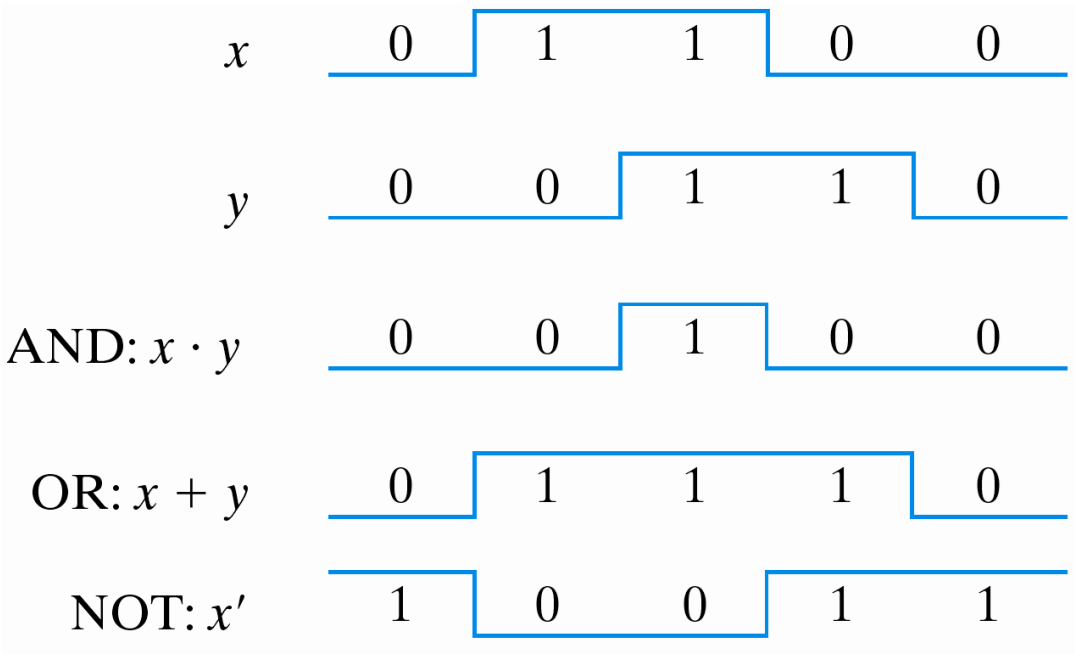
x	x'
0	1
1	0

Logic Gates

- Symbols and Input-Output Signals for Logic gates



(a) Two-input AND gate (b) Two-input OR gate (c) NOT gate or inverter



Operators in Verilog

Table 8.1
Verilog 2001 HDL Operators

Operator Type	Symbol	Operation Performed
Arithmetic	+	addition
	–	subtraction
	*	multiplication
	/	division
	%	modulus
	**	exponentiation
Bitwise or Reduction	~	negation (complement)
	&	AND
	|	OR
	^	exclusive-OR (XOR)
Logical	!	negation
	&&	AND
	||	OR
Shift	>>	logical right shift
	<<	logical left shift
	>>>	arithmetic right shift
	<<<	arithmetic left shift
	{ , }	concatenation
Relational	>	greater than
	<	less than
	==	equality
	!=	inequality
	===	case equality
	!==	case inequality
	>=	greater than or equal
	<=	less than or equal

Copyright ©2012 Pearson Education, publishing as Prentice Hall

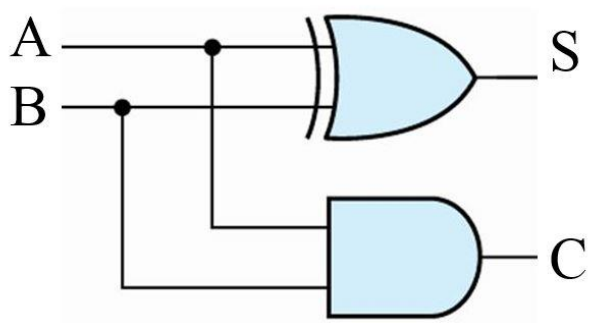
Table 8.2
Verilog Operator Precedence

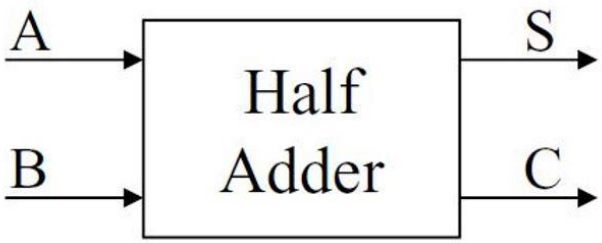
+ – ! ~ & ~& | ~ | ^ ~^ ^~ (unary)	Highest precedence
**	↓
* / %	
+ – (binary)	
<< >> <<< >>>	
< < = > >=	
== != === !==	
& (binary)	
^ ^~ ~^ (binary)	
| (binary)	
&&	
||	
?: (conditional operator)	
{ } { { } }	Lowest precedence

Copyright ©2012 Pearson Education, publishing as Prentice Hall

Use parentheses to enforce your priority

Example 1: Half Adder (Behavioral level modeling)



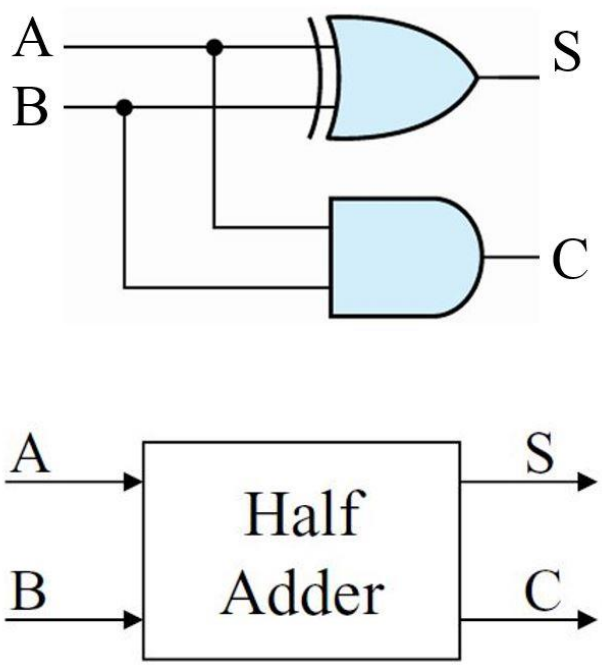


$S = A \oplus B$

$C = AB$

```
module half_adder (S, C, A, B);
    output reg S, C;
    input A, B;

    always @ (A or B)
    begin
        S = A ^ B;
        C = A & B;
    end

endmodule
```

Example 2: Half Adder (Dataflow modeling)



$S = A \oplus B$

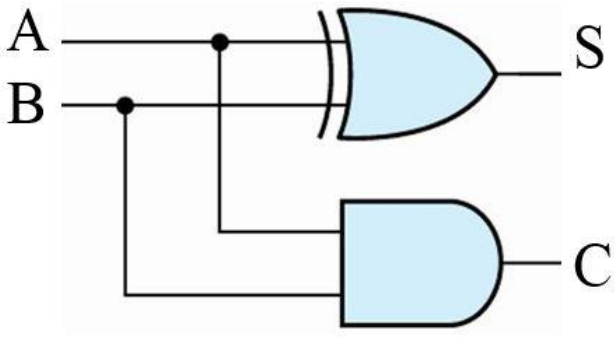
$C = AB$

```
module half_adder (S, C, A, B);
    output S, C;
    input A, B;

    assign S = A ^ B;
    assign C = A & B;

endmodule
```

Verilog Structural Model (Gate Level)

- Built-in gate primitives:
  - **and, nand, nor, or, xor, xnor, buf, not, bufif0, bufif1, notif0, notif1**
- Usage:
  - nand (out, in1, in2); 2-input NAND without delay
  - and #2 (out, in1, in2, in3); 3-input AND with 2 t.u. delay
  - not #1 N1(out, in); NOT with 1 t.u. delay and instance name
  - xor X1(out, in1, in2); 2-input XOR with instance name
- Write them inside module, outside procedures

Example 3: Half Adder (Structural level modeling)

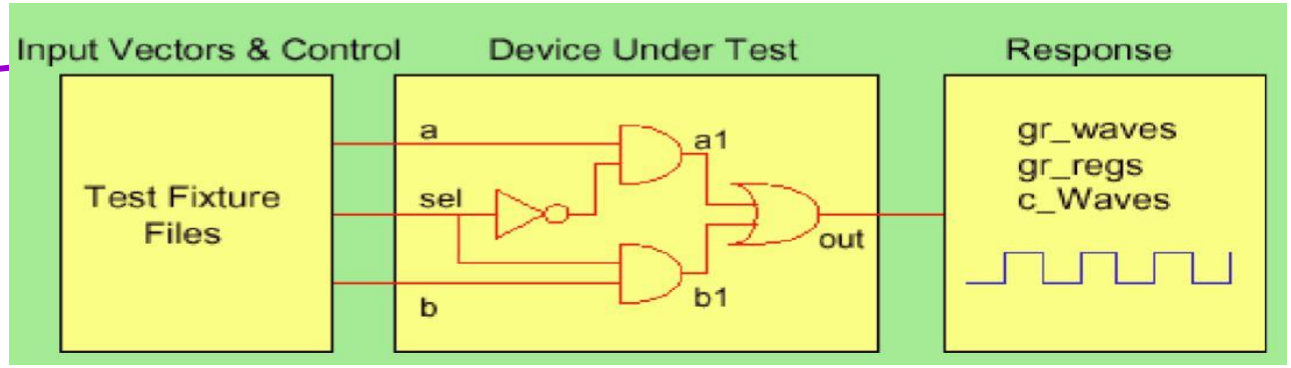


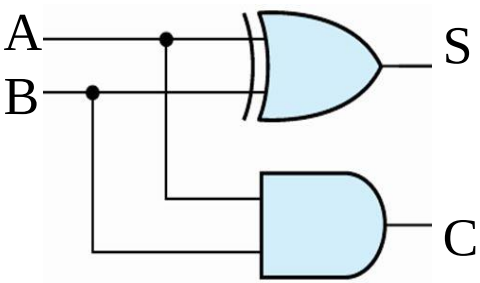
Assuming:

- XOR: 2 t.u. delay
- AND: 1 t.u. delay

```
module half_adder (S, C, A, B);
    output S, C;
    input A, B;

    xor #2 (S, A, B);
    and #1 (C, A, B);

endmodule
```

Testbench

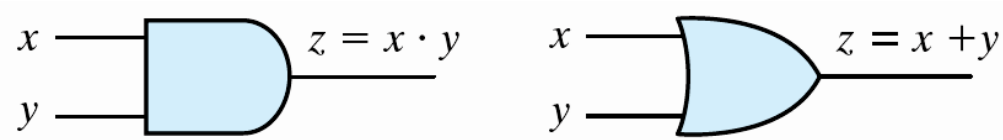





```
`timescale 1ns / 1ps
module t_half_adder();
  reg A;
  reg B;
  wire S;
  wire C;

  half_adder UUT ( .A(A), .B(B), .S(S), .C(C) );

  initial begin
     A=1'b0; B=1'b0;
     #10  A=1'b0; B=1'b1;
     #10  A=1'b1; B=1'b0;
     #10  A=1'b1; B=1'b1;
     #10  $finish;
  end
endmodule
```

Exercise 1: Vivado Block Design

- 目標:
  - 使用Vivado的Block Design功能拉出輸入輸出都是1bit的two-input OR閘
- 要求:
  - 必須完成synthesis並且模擬結果正確
  - Testbench (TB) 檔的測試內容必須包含A=0、B=0，A=0、B=1，A=1、B=0，A=1、B=1



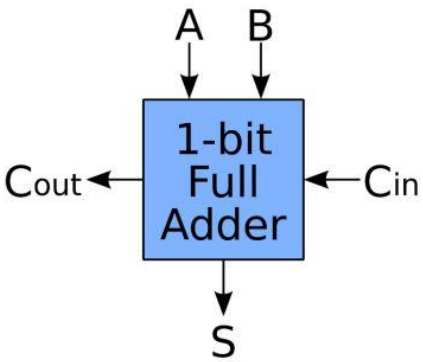
(a) Two-input AND gate (b) Two-input OR gate

OR

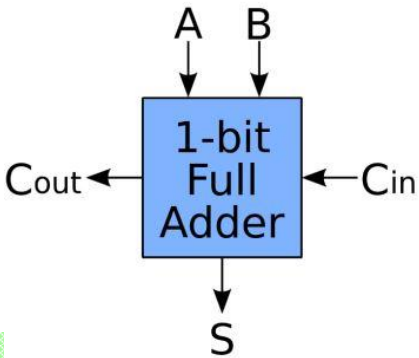
x	y	$x + y$
0	0	0
0	1	1
1	0	1
1	1	1

Exercise 2: Verilog

- 目標:
  - 使用Verilog將上課的範例full_adder從Behavior level modeling修改成Dataflow modeling
- 要求:
  - 必須完成synthesis並且模擬結果正確




```
// behavioral level modeling

module fulladd(sum, c_out, a, b, c_in);
output  reg  sum, c_out;
input a, b, c_in;

always @ (a or b or c_in)
    {c_out, sum} = a + b + c_in;

endmoudle
```




```
// dataflow modeling

module fulladd(sum, c_out, a, b, c_in);
output sum, c_out;
input a, b, c_in;

assign {c_out, sum} = a + b + c_in;

endmoudle
```